MERGE SORT

A DIFFERENZA DEGLI APPROCCI USATI PER INSERTION E SELECTION SORT, IL MERGE SORT USA LA RICORSIONE, SFRUTTANDO IL PARADIGMA DEL DIVIDI ET IMPERA.

IL PROCESSO SI ARTICOLA IN 3 FASI:

1. DIVIDI: DIVIDE LA SEQUENZA DI n ELEMENTI A METÀ, CALCOLANDO L'INDICE CENTRALE, OTTENENDO 2 SEQUENZE DI $\frac{n}{2}$ ELEM.
2. IMPERA: ORDINA RICORSIVAMENTE LE DUE SOTTOSEQUENZE, CHIAMANDO MERGESORT SU CIASCUNA. IL CASO BASE SI RAGGIUNGE CON UNA SEQUENZA DI LUNGHEZZA 1.
3. COMBINA: FONDE LE DUE SOTTOSEQUENZE ORDINATE, TRAMITE LA PROCEDURA AUSILIARIA MERGE.

```
MERGESORT(A, start, end)
  IF START < END
    // 1. DIVIDE
    MID = FLOOR((START + END)/2)
    // 2. IMPERA
    MERGESORT(A, START, MID)
    MERGESORT(A, MID+1, END)
    // 3. COMBINA
    MERGE(A, START, MID, END)
```

PROCEDURA MERGE

LA PROCEDURA MERGE FONDE I DUE SOTTOARRAY ORDINATI APPOGGIANDOSI A UN ARRAY AUSILIARIO.

```
MERGE(A, START, MID, END)
    CURR_START = START  // INDICE PER SCORRERE METÀ SX
    CURR_END = MID+1    // INDICE PER SCORRERE METÀ DX
    CURR_AUX = START    // INDICE PER SCORRERE ARRAY AUSILIARIO
    // CONFRONTO E FUSIONE DELLE METÀ
    WHILE (CURR_START ≤ MID && CURR_END ≤ END)
        IF A[CURR_START] ≤ A[CURR_END]
            AUX_A[CURR_AUX] = A[START]
            CURR_START = CURR_START + 1
        ELSE
            AUX_A[CURR_AUX] = A[CURR_END]
            CURR_END = CURR_END+1
        CURR_AUX = CURR_AUX + 1
    //GESTIONE DEL RESIDUO: SVUOTA EVENTUALE PARTE SX RIMASTA
    WHILE (CURR_START ≤ MID)
        AUX_A[CURR_AUX] = A[CURR_START]
        CURR_START = CURR_START +1
        CURR_AUX = CURR_AUX +1
    //GESTIONE DEL RESIDUO: SVUOTA EVENTUALE PARTE DX RIMASTA
    WHILE (CURR_END ≤ END)
        AUX_A[CURR_AUX] = A[CURR_END]
        CURR_END = CURR_END+1
        CURR_AUX = CURR_AUX +1
```

CORRETTEZZA

DIMOSTRARE LA CORRETTEZZA DELL'ALGORITMO È NECESSARIO PER NON CADERE IN POTENZIALI LOOP INFINITI. C'È QUINDI BISOGNO CHE LE ISTANZE FIGLIE GENERATE DAL TAGLIO IN MID SIANO STRETTAMENTE PIÙ PICCOLE DELL'ISTANZA PADRE DI DIMENSIONE $n = END - START + 1$.

SAPENDO CHE $MID = \left[\frac{START + END}{2}\right]$ E CHE PER LE PROPRIETÀ DELLA PARTE INFERIORE VALE $[x] \leq x$, SI DIMOSTRA CHE:

- PRIMA SOTTOSEQUENZA: $END - START + 1 > \frac{START + END}{2} - START + 1$, QUINDI $END > START$ (VERO PER LA CONDIZIONE DI GUARDIA IF (START < END))
- IL TAGLIO RIDUCE SEMPRE L'INPUT: NEL CASO LIMITE IN CUI L'ARRAY HA 2 ELEMENTI ($END = START + 1$), MID COLLASSA SU START. LE SUCCESSIVE CHIAMATE AVRANNO DIMENSIONE 1 E 1, COLPENDO I CASI BASE E TERMINANDO LA RICORSIONE

ANALISI DELLA COMPLESSITÀ

IL TEMPO DI ESECUZIONE DI MERGE SORT SI FORMALIZZA IN UN'EQUAZIONE DI RICORRENZA STANDARD PER GLI ALBERI BILANCIATI

- CASO BASE $n \leq 1$ SI RISOLVE IN TEMPO COSTANTE $\Theta(1)$
- PER $n > 1$, LA FASE DI DIVIDE COSTA $\Theta(1)$, QUELLA DI CONQUER COSTA $2T(n/2)$, QUELLA DI COMBINE $\Theta(n)$ PER SCORRERE GLI ELEMENTI

L'EQUAZIONE DI RICORRENZA È:

$$T_{MS}(n) = \begin{cases} \Theta(1) & \text{SE } n \leq 1 \\ 2T_{MS}(n/2) + \Theta(n) & \text{SE } n > 1 \end{cases}$$

RISOLUZIONE

$a = 2 \qquad b = 2 \qquad g(n) = \Theta(n)$

$c_{l_0} = n$

$c_{l_1} = 2 \frac{n}{2}$

$q = \frac{n}{n} = 1$

LIMITE INFERIORE Ω:

$h = \log_2 n$

$\Omega(n \log_2 n)$

LIMITE SUP O

$h = \log_2 n$

$O(n \log_2 n)$

$\Rightarrow T(n) = \Theta(n \log_2 n)$